

Lab1: DataLab

方碧珩 24307110160

1 符号检查及最终得分

在终端中执行`./check_ops.py bits.c`，结果表明通过符号检查。

```
bhfang06@LAPTOP-M13573N0:~/ICS/lab1-datalab-24307110160$ ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 15/18 operators
secondLowestZeroBit: 8/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 21/32 operators
isBetweenEitherOrder: 39/48 operators
mul5Sat: 22/30 operators
classifyAdd3: 51/52 operators
floatScaleThreeHalves: 45/60 operators
floatRoundEven: 30/65 operators
float_i2f: 24/40 operators
bitCount: 36/40 operators
bitReverse: 34/34 operators
All 19 functions passed operator checks.
```

图 1: 符号检查

继续在终端中执行`./btest`，结果表明本次实验得分为 110/110。

```
bhfang06@LAPTOP-M13573N0:~/ICS/lab1-datalab-24307110160$ ./btest
Score  Rating  Errors  Function
1      1        0      signMask
2      2        0      bitXor
3      3        0      negativePart
4      4        0      copyByteWithin
4      4        0      logicalShift
4      4        0      swapNibblePairs
4      4        0      secondLowestZeroBit
5      5        0      oddParity
5      5        0      rotateRightBits
5      5        0      roundEvenPow2
5      5        0      midpointTowardFirst
7      7        0      isBetweenEitherOrder
7      7        0      mul5Sat
7      7        0      classifyAdd3
7      7        0      floatScaleThreeHalves
10     10        0      floatRoundEven
10     10        0      float_i2f
10     10        0      bitCount
10     10        0      bitReverse
Total points: 110/110
```

图 2: 实验得分

2 函数设计思路

2.1 P1 signMask

本题要求实现的函数功能为：返回一个仅最高有效位为 1 的掩码（即 0x80000000）。

思路：利用左移运算，将 1 左移 31 位。

2.2 P2 bitXor

本题要求实现的函数功能为：仅用 $\sim$ 和 $\&$ 实现 $x \wedge y$ 。

思路： $x \wedge y = \begin{cases} 0, & (x, y) = (0, 0) \text{ 或 } (1, 1), \\ 1, & (x, y) = (1, 0) \text{ 或 } (0, 1). \end{cases}$

先考虑 $x \wedge y = 1$ 的情况：x,y 必有一个为 1，即 $x|y = 1$ ，此时另一个量必须为 0，即 $x \& y = 0$ 。

再考虑 $x \wedge y = 0$ 的情况：x,y 要么同为 1，要么同为 0，此时 $x|y = 1, x \& y = 1$ 或 $x|y = 0, x \& y = 0$ 。

由此可以构造出函数： $x \wedge y = (x|y) \& \sim(x \& y)$

又因为题目仅允许使用 $\sim$ 和 $\&$ ，于是将 $x|y$ 作如下替换：

$x | y = \sim(\sim x \& \sim y)$

最终得到： $x \wedge y = (\sim(\sim x \& \sim y)) \& \sim(x \& y)$

2.3 P3 negativePart

本题要求实现的函数功能为：当 x 为负数时，返回它的相反数；否则，返回 0。

思路：首先将 x 右移 31 位，即可得到其符号位 sign。若 $sign = 1$ ，则 x 非负，返回 0；若 $sign = 1$ ，则 x 为负数，返回 -x。已知 $-x = \sim x + 1$ ，故可以构造返回值为 $(\sim x + 1) \& sign$ ；

2.4 P4 copyByteWithin

本题要求实现的函数功能为：将整数 x 中编号为 src 的字节复制到编号为 dst 的字节，同时保持其余字节不变。字节编号从低位到高位依次为 0-3。

思路：首先将 src 和 dst 乘以 8，得到对应字节在 32 位整数中的起始位数。利用算术右移和掩码 0xFF 提取源字节：

```
1 int byte = (x >> (src << 3)) & 0xFF;
```

随后构造目标字节掩码 $0xFF \ll (dst \ll 3)$ 。先用该掩码的按位取反将目标字节清零，再将提取出的源字节左移到 dst 对应的位置，最后利用按位或运算写回：

```
1 return (x & ~mask) | (byte << dstShift);
```

由于源字节在右移后立即与 0xFF 相与，因此即使 x 为负数，算术右移产生的符号扩展位也会被清除。

2.5 P5 logicalShift

本题要求实现的函数功能为：实现逻辑右移。

思路：逻辑右移与算数位移的区别在于：算数位移用符号位将高位补齐，而逻辑位移用 0 将高位补齐，对于负数来说两种运算得到不同的结果。因此本题要考虑的核心问题是如何在高位补 0。

我们构造一个掩码 mask 来讲高位补齐：首先通过 $1 \ll 31$ 得到一个首位为 1、后 31 位为 0 的数；然后通过算数右移 $(1 \ll 31) \gg n$ 令其高 n 位均为 1，即：111...1000...0；最后将其取反，即得我们所需的掩码：000...0111...1，其中高 n 位为 0。

令返回结果为 $(x \gg n) \& \text{mask}$ ，即可实现逻辑右移。

2.6 P6 swapNibblePairs

本题要求实现的函数功能为：交换 x 的每个字节里的高 4 位与低 4 位。

思路：利用掩码分别取出每个字节里的高四位与低四位，最后使用或运算实现交换。

若要提取每个字节的后四位，即令掩码的相应位置为 1 而其他位置为 0，即： $\text{maskLow} = 0x0F0F0F0F$ 。由于 DataLab 对于常数取值的限制，我们从 $0x0F$ 出发，通过左移运算得到 $0x0F0F0F0F$ ：首先由 $0x0F|0x0F \ll 8$ 得到长度为 2 个字节的 $0x0F0F$ ，然后同理由 $0x0F0F|0x0F0F \ll 16$ 得到长度为 4 个字节的 $0x0F0F0F0F$ ，即 maskLow 。

而 $\text{maskHigh} = 0xF0F0F0F0$ ，注意到可由 maskLow 左移得到，即： $\text{maskHigh} = \text{maskLow} \ll 4$ ；

接下来，我们只需提取每个字节的低四位，左移 4 位，提取每个字节的高四位，右移 4 位即可。但是在右移时若使用算数右移会在高位补 0 导致报错，因此必须使用逻辑右移，参考 P5 的代码，有：

```
1 int shifted = (x & maskHigh) >> 4;
2 logicMask = ~(((1 << 31) >> 4) << 1);
3 shifted = shifted & logicMask;
```

最后使用或运算将二者整合即可： $\text{return } ((x \& \text{maskLow}) \ll 4) | \text{shifted}$;

2.7 P7 secondLowestZeroBit

本题要求实现的函数功能为：返回一个掩码，标记出第二低位 0 的位置。

思路：对于任意整数 x ， $\sim x \& (x + 1)$ 可以提取 x 中最低位的 0。其原因是 $x + 1$ 会将最低位 0 变为 1，并将其右侧连续的 1 变为 0，而与 $\sim x$ 相与后仅保留这一位置。

2.8 P8 oddParity

本题要求实现的函数功能为：判断 x 的二进制表示中 1 的个数是奇数还是偶数。若为偶数，则返回 1；反之则返回 0。

思路：不断将高位与低位异或，直到只剩一位。

使用异或运算的好处是： $1 \wedge 1 = 0$ ， $1 \wedge 0 = 1$ ，即在每次运算中，以两个 1 为一组进行消除。当消除至只剩一位时，若为 1，则说明原数有奇数个 1，应返回 0；若为 0，则说明原数有偶数个 1，应返回 1。（返回值对异或运算的结果取反即可）

现在只需考虑如何进行异或运算使最终结果只剩一位。我们可以在每轮异或运算中将 x 分割为长度相等的两段，将两段进行异或运算，在每轮运算过后， x 剩余有效位数的长度减半，因此总共需要进行五轮运算使 x 被消除得只剩一位。

最后提取 x 的最后一位并取反，即得返回值 $!(x \& 1)$ 。

2.9 P9 rotateRightBits

本题要求实现的函数功能为：将 x 右移 n 位，因右移而消失的低 n 位按顺序放在高 n 位。

思路：将 x 的高 $(32-n)$ 位右移 n 位，低 n 位左移 $(32-n)$ 位，然后利用或运算合并。

由于本题不允许使用减号，故需要利用下式进行替换： $-n = \sim n + 1$

本题的右移同样必须使用逻辑右移，设置掩码 $\text{mask} = \sim(((1 \ll 31) \gg n) \ll 1)$

完成以上准备后，分别通过左移和右移运算得到 x 的高位与低位：

```
1 int right = (x >> n) & mask;
2 int left = x << shift;
```

最后，使用或运算将两个部分合并，得到返回值。

2.10 P10 roundEvenPow2

本题要求实现的函数功能为：将非负整数 x 舍入到最接近的 2^n 的整数倍；若 x 恰好位于两个相邻倍数的中点，则选择商为偶数的那个倍数，即采用 round-to-nearest-even 规则。

思路：将 x 写成

$$x = q \cdot 2^n + r, \quad 0 \leq r < 2^n.$$

其中 $q = x \gg n$ ，而余数可通过低 n 位掩码得到。由于不能使用减法，利用 $\text{mask} = (1 \ll n) + \sim 0$ ；构造 $2^n - 1$ ，进而有 $\text{rem} = x \& \text{mask}$ 。中点为 2^{n-1} 。

若余数大于中点，则商需要加 1；若余数小于中点，则保持不变；若余数恰好等于中点，则仅在当前商 q 为奇数时加 1，从而使最终商为偶数。最后将修正后的商左移 n 位，即得到所需的 2^n 的倍数。

2.11 P11 midpointTowardFirst

本题要求实现的函数功能为：在不发生整数溢出的前提下计算 x 与 y 的数学中点。当中点恰好位于两个整数之间时，选择更靠近第一个参数 x 的那个整数。

思路：直接计算 $(x+y)/2$ 可能在 x 与 y 同号且绝对值较大时溢出，因此采用恒等式

$$\left\lfloor \frac{x+y}{2} \right\rfloor = (x \& y) + ((x \wedge y) \gg 1).$$

这里 $(x \& y)$ 保存两数共同为 1 的位， $(x \wedge y) \gg 1$ 则贡献其余差异部分的一半，从而避免直接求和。

当 x 与 y 奇偶性不同时，真实中点位于两个整数之间，此时 $(x \wedge y) \& 1$ 为 1。上述表达式给出向负无穷方向的整数，因此仅当 $x > y$ 且存在半整数情形时再加 1，即可使结果向第一个参数 x 靠近。代码中通过符号位以及差值的符号共同判断 $x > y$ ，从而避免比较过程中有符号溢出带来的错误。

2.12 P12 isBetweenEitherOrder

本题要求实现的函数功能为：判断 x 是否位于以 a 和 b 为端点的闭区间内，且 a 、 b 的输入顺序可以任意。若在区间内返回 1，否则返回 0。

思路：需要判断

$$a \leq x \leq b \quad \text{或} \quad b \leq x \leq a.$$

困难在于不能直接使用比较运算符，而且简单地通过 $x-a$ 的符号判断大小关系会在有符号减法溢出时失效。

因此先用 $u + \sim v + 1$ 实现 $u - v$ ，再结合 u 、 v 与差值三者的符号位进行溢出修正。对于 $u - v$ ，其真实符号可以写成：当 u 与 v 同号时采用差值符号；当二者异号时直接采用 u 的符号。 $\text{diff} \wedge ((u \wedge v) \& (u \wedge \text{diff}))$ 正是对减法结果符号的修正。分别得到 aLeX 、 xLeA 、 bLeX 、 xLeB 后，再组合两个可能的端点顺序：

```
1 return (aLeX & xLeB) | (bLeX & xLeA);
```

即可完成判断。

2.13 P13 mul5Sat

本题要求实现的函数功能为：计算 $x \times 5$ ，若溢出，则根据 x 的正负返回 INT_MAX 或 INT_MIN 。

思路：

1. 计算 $x \times 5$:

$$mul2 = x \ll 1$$

$$mul4 = mul2 \ll 1$$

$$mul5 = mul4 + x$$

2. **判断溢出:** 溢出会导致符号位变化。对有符号整数运算来说, 若乘法结果的符号与原数不同, 则溢出。故分别判断 `mul2`, `mul4`, `mul5` 的计算是否发生溢出:

```
1      int overflow2 = signX ^ signMul2;
2      int overflow4 = signX ^ signMul4;
3      int overflow5 = signX ^ signMul5;
```

若其中任意一步发生溢出, 则表明 `mul5` 的计算发生溢出。故整体的 `overflow = overflow2 | overflow4 | overflow5`;

3. **生成结果:** 若发生溢出, 则根据 `x` 的符号判断最终的返回值, 若 $x \geq 0$, 则返回 `INT_MAX`, 反之则返回 `INT_MIN`。`x` 右移 31 位得 `x` 的符号位 `signX`。故发生溢出时的返回值 `resultIfOverflow = (signX & INT_MIN) | (~signX & INT_MAX)`;

若不发生溢出, 则返回 `mul5`, 故最终的返回值为 `(overflow & resultIfOverflow) | (~overflow & mul5)`¹

2.14 P14 classifyAdd3

本题要求实现的函数功能为: 判断数学意义上的精确和 $x + y + z$ 是否超出 32 位有符号整数范围。若大于 `INT_MAX` 返回 1, 若小于 `INT_MIN` 返回 -1, 否则返回 0, 并且不能使用更宽的整数类型。

思路: 由于三个 32 位有符号整数相加的真实结果可能超过 32 位, 因此不能只观察最终 32 位结果的符号。实现中先依次计算

```
1  s1 = x + y;
2  s2 = s1 + z;
```

同时分别计算两次 32 位加法产生的最高位进位 `c1` 和 `c2`。对于无符号加法 $a + b = s$, 进位由

```
1  ((a & b) | ((a | b) & ~s))
```

的最高位得到。

另一方面, 若把 x, y, z 按补码解释, 则真实数学和可以写成

$$S = s_2 + 2^{32}(c_1 + c_2 - s_x - s_y - s_z),$$

其中 $s_x, s_y, s_z \in \{0, 1\}$ 分别表示三个操作数的符号位。因此定义

$$k = c_1 + c_2 - s_x - s_y - s_z.$$

根据 k 的取值以及低 32 位结果 s_2 的符号, 可以判断真实结果是否落在有符号 32 位范围之外: 若整体高于 `INT_MAX` 返回 1, 低于 `INT_MIN` 返回 -1, 否则返回 0。这样无需任何 64 位数据类型即可完成精确分类。

¹P13 参考了 https://zhuanlan.zhihu.com/p/680186261?share_code=wYrm4VrFWH8L&utm_psnI961110992411923340 中 `satMul3` 部分的内容

2.15 P15 floatScaleThreeHalves

本题要求实现的函数功能为：给定单精度浮点数 f 的位级表示，返回 $f \times \frac{3}{2}$ 的位级表示，并采用 round-to-nearest-even 舍入，保留 +0 与 -0 的符号；对于 NaN 原样返回。

思路：首先利用掩码分别提取符号位 `sign`、8 位指数 `exp` 和 23 位尾数 `frac`。若 `exp == 0xFF`，输入为 NaN 或无穷大，直接返回原值。

对于非规格化数，数值没有隐含的最高位 1，因此直接对尾数计算 $3 \times \text{frac}$ ，再除以 2。除以 2 时若被舍弃部分恰好位于中点，则依据保留结果最低位的奇偶性完成 round-to-nearest-even。若舍入后的结果进入第 23 位以上，则说明数值从非规格化区跨入最小规格化数区，需要相应设置指数位。

对于规格化数，先恢复隐含的最高位：

```
1 sig = 0x800000 | frac;
```

再计算 `three = sig + (sig << 1)`。根据结果的最高有效位位置，选择右移 1 位或 2 位完成除以 2 与规格化，并利用被舍弃的低位执行 round-to-nearest-even。若尾数舍入产生新的进位，则继续右移并令指数加 1。最后若指数达到 255，则结果溢出为同符号的无穷大，否则重新拼接 `sign`、`exp` 和 `frac`。

2.16 P16 floatRoundEven

本题要求实现的函数功能为：将由 `uf` 表示的单精度浮点数四舍五入到最接近的整数；恰好在两个整数中间时舍入到偶数。若结果为 0，则保留输入的符号；NaN 和无穷大原样返回。

思路：首先提取 `sign`、绝对值 `mag` 与指数 `exp`，并单独处理特殊值。若 `exp == 0xFF`，则为 NaN 或无穷大，直接返回。对于 $|f| < 0.5$ ，舍入结果为带原符号的 0；当 $|f| = 0.5$ 时，根据 ties-to-even 规则同样舍入为 0；而 $0.5 < |f| < 1$ 时舍入为 ± 1 。

对于 $|f| \geq 1$ 的规格化数，令真实指数 $e = \text{exp} - 127$ 。当 $e \geq 23$ 时，尾数已经没有小数位，原值本身就是整数，可直接返回。若 $0 \leq e < 23$ ，则低 $23 - e$ 位属于小数部分。构造

```
1 mask = (1u << (23 - e)) - 1u;
```

提取余数 `rem`，并用 `base = mag & ~mask` 清除所有小数位。中点对应 `half = 1u << (22 - e)`。若 `rem` 大于 `half`，则向上舍入；若 `rem` 等于 `half`，则仅当整数部分最低位为 1 时进位，使最终整数为偶数。最后重新加上原始符号位即可。

2.17 P17 float_i2f

本题要求实现的函数功能为：把一个 32 位整数 `int x` 转换成单精度浮点数（IEEE 754 格式）的位级表示（unsigned int）。

思路：

1. **特殊情况 $x=0$ ：** `return 0`;
2. **提取符号并计算绝对值：**若 $x < 0$ ，则 `sign = 0x80000000`，否则为 0。 $x < 0$ 时，`x == 0x80000000` 的绝对值须单独讨论以防止溢出，其余情况 `absX = -x`。
3. **找出最高有效位并计算指数：**找到最高的 1 的位置，并将其左移到第 31 位；同时设置 `shift` 用于计数，每左移一次，指数升高 1。

$$\text{exp} = (31 - \text{shift}) + \text{bias} = 31 - \text{shift} + 127 = 158 - \text{shift}$$

4. **提取小数部分 frac**: 去掉最高位, 取高 23 位; 同时用 `remainder` 保存其余 8 位用于舍入。根据 IEEE 规则“舍入到偶数”进行小数部分的舍入, 若尾数进位导致多出一位溢出, 则清除溢出位并使指数 +1。
5. 最后利用或运算与左移操作将 `sign`、`exp`、`frac` 三个部分合并。

2.18 P18 bitCount

本题要求实现的函数功能为: 统计一个 32 位整数中二进制 1 的个数。

思路: 把 32 位整数拆成多个小组 (2 位一组/4 位一组/8 位一组/16 位一组), 然后逐步累加各种分组下每组中 1 的数量, 直到得出总和。

首先通过掩码 `0101...0101` (掩码通过 `0x55` 移位获得) 提取 `x` 的偶数位, 然后将奇数位右移一位并与偶数位进行与运算, 得出每两位中 1 的个数, 分别储存在各两位小组中。

同理, 利用掩码 `0011...0011` 将两位的小组两两合并, 继续得出每四位中 1 的个数。

依此类推, 合并结果至 8 位、16 位、32 位, 即可得出最终结果。²

2.19 P19 bitReverse

本题要求实现的函数功能为: 反转一个 32 位整数的所有位。

思路: 采用分层交换的方法逐步完成位反转。若直接一次性移动每一位, 操作数会远超题目限制; 而位反转可以拆解为一系列固定宽度分组的交换: 先交换相邻的 8 位组, 再交换 4 位组、2 位组和 1 位组, 最后交换高、低 16 位。

首先构造掩码 `0x00FF00FF`:

```
1 m = 0xFF | (0xFF << 16);
2 x = ((x >> 8) & m) | ((x & m) << 8);
```

这样即可交换每个 16 位区域内的两个 8 位组。之后无需重新使用大常数, 而是连续由已有掩码生成

`0x0F0F0F0F`, `0x33333333`, `0x55555555`,

分别完成 4 位、2 位和 1 位分组的交换:

```
1 m = m ^ (m << 4);
2 m = m ^ (m << 2);
3 m = m ^ (m << 1);
```

每一步都采用

```
1 ((x >> k) & m) | ((x & m) << k)
```

的形式。虽然题目规定右移为算术右移, 但右移结果随即与对应掩码相与, 因此符号扩展产生的高位 1 会被清除, 不影响交换结果。

最后构造低 16 位掩码 `0x0000FFFF`, 交换高、低 16 位:

```
1 m = 0xFF | (0xFF << 8);
2 x = ((x >> 16) & m) | (x << 16);
```

经过 8、4、2、1、16 位五级交换后, 32 位顺序恰好完全反转。该实现共使用 34 个允许的运算符, 满足题目的 Max ops 限制。

²P18 参考了 <https://youtu.be/0rdRRURz6IQ?si=zAWvJgmPJmXi4jjO> 中视频的内容。